UT 03.1 · REACT CON TYPESCRIPT

Renderizado condicional y listas

Mostrar lo que toca, cuando toca, y pintar colecciones de datos con tipos seguros.

```
{datos.length > 0
  ? <Lista datos={datos} />
  : <Vacio />}
```

OBJETIVOS

Qué vas a ser capaz de hacer

01

Mostrar y ocultar

Con if, ternarios, && y mapas de valores.

02

Modelar estados

Uniones discriminadas: cargando, error, datos.

03

Pintar listas

map() y componentes con props tipadas.

04

Transformar datos

Filtrar, ordenar y agrupar sin mutar.

PUNTO DE PARTIDA

Dos necesidades en casi cualquier aplicación

Contenido condicional

- Mensaje de error tras un envío fallido
- Indicador de carga mientras llegan datos
- Menú lateral que se despliega al pulsar
- Campos extra si el usuario es empresa

Listas de datos

- Productos de una tienda online
- Carrito y pedidos
- Entradas y comentarios de un blog
- Enlaces del menú de navegación

Lo habitual: una lista que se filtra, avisa si está vacía y cambia el aspecto de cada elemento según su estado.

BLOQUE 1

Renderizado condicional

Si, no, o depende: elegir qué se pinta en cada momento.

RENDERIZADO CONDICIONAL

Un botón y un texto oculto

```
const [mostrar, setMostrar] = useState(false);

let texto = "";
if (mostrar) {
  texto = "Al continuar, aceptas…";
}

return (
  <section>
    <button onClick={() => setMostrar(true)}>Ver</button>
    <p>{texto}</p>
  </section>
);
```

Problema: el p siempre existe

Sin texto, queda un elemento vacío en el DOM.

Con TypeScript

useState(false) infiere boolean: no hace falta escribir el tipo.

RENDERIZADO CONDICIONAL

El JSX también cabe en una variable

```tsx
import { useState, type ReactNode } from "react";

// ReactNode: todo lo que React sabe pintar
let parrafo: ReactNode = null;
if (mostrar) {
  parrafo = <p>Al continuar, aceptas…</p>;
}

return (
  <section>
    <button …>Ver</button>
    {parrafo}
  </section>
);
```

Ahora sí

Con null no se pinta nada; con JSX se inserta el p completo.

Ojo con JSX.Element

Con React 19 ya no existe como tipo global: usa ReactElement o ReactNode.

Qué hace React con cada valor

Valor en las llaves	¿Se pinta?	Comentario
true, false	No	Permiten escribir cond && <A />
null, undefined	No	La forma de «no pintar nada»
0, NaN	Sí: muestra 0	Origen de la trampa del cero
"" (cadena vacía)	Nada visible	Solo un nodo de texto vacío
"Hola", 42	Sí	Se muestran como texto
Un array	Sí, cada elemento	Base para pintar listas

TÉCNICA 1

El ternario: dos alternativas

```
// if / else
let a = 1;
if (condicion) { a = 2; }

// ternario
const a = condicion ? 2 : 1;

// en JSX, con botón que alterna
<button onClick={() => setVer((v) => !v)}>
  {ver ? "Ocultar" : "Mostrar"} resumen
</button>
{ver ? <p>Condiciones…</p> : null}
```

Mejora sobre el original

El botón alterna con la forma funcional de setState y aria-expanded.

null es seguro

Como alternativa no pinta nada.

TÉCNICA 1

Ternarios anidados: mejor un if

✗ Difícil de leer

```
const parrafo = !mostrar
  ? null
  : otraCondicion
    ? <p>Condiciones…</p>
    : null;
```

✓ Claro con return temprano

```
function textoCondiciones(): ReactNode {
  if (!mostrar || !otraCondicion) {
    return null;
  }
  return <p>Condiciones…</p>;
}
```

> Regla práctica: más de una condición, usa if y extrae un componente.

TÉCNICA 2

El operador &&: cortocircuito

```
// el else (: null) no aporta nada
{mostrarDetalles ? <h1>Detalles</h1> : null}

// más corto
{mostrarDetalles && <h1>Detalles</h1>}

// 1 === 1 && "Hola" → "Hola"
// false && "Hola" → false (no se pinta)
```

Cómo funciona

Si lo de la izquierda es verdadero, devuelve lo de la derecha; si no, devuelve lo de la izquierda.

CUIDADO

La trampa del cero

✗ Condición numérica

```
{mensajes.length &&
  <p>Tienes {mensajes.length} avisos</p>}

// con 0 avisos, React pinta: 0
```

✓ Condición booleana

```
{mensajes.length > 0 &&
  <p>Tienes {mensajes.length} avisos</p>}

// o: !!valor, Boolean(valor), if + null
```

TypeScript no lo detecta: number && JSX es válido. Hay que saberlo.

TÉCNICA 3

Salida temprana: return null

```tsx
interface PropsAvisos {
  mensajes: string[];
}

function Avisos({ mensajes }: PropsAvisos) {
  if (mensajes.length === 0) {
    return null;
  }
  return <p>Tienes {mensajes.length} avisos</p>;
}
```

Cuándo usarla

Cuando la condición afecta al componente completo.

Ventaja

Ningún ternario ni &&: el caso feliz queda al final, sin anidar.

TÉCNICA 4

Mapas de valores con Record

```ts
type Pais = "de" | "us" | "uk";

const idiomas: Record<Pais, string> = {
  de: "de-DE",
  us: "en-US",
  uk: "en-GB",
};

function SelectorIdioma({ pais }: { pais: Pais }) {
  return <p>Idioma: {idiomas[pais]}</p>;
}
```

Tipo completo

Si añades un país y olvidas el mapa, no compila.

Con pais: string

Error TS7053: un string cualquiera no sirve de índice.

TÉCNICA 5

Estados de pantalla: unión discriminada

```
type EstadoPeticion<T> =
  | { estado: "cargando" }
  | { estado: "error";
      mensaje: string }
  | { estado: "correcto";
      datos: T };
```

cargando
Solo la etiqueta de estado.

error
Además existe mensaje.

correcto
Además existen los datos.

Imposible representar «cargando y con error a la vez»: el estado queda bien definido.

TÉCNICA 5

switch exhaustivo con never

```
switch (peticion.estado) {
  case "cargando":
    return <p>Cargando…</p>;
  case "error":
    return <p>Error: {peticion.mensaje}</p>;
  case "correcto":
    return <Lista datos={peticion.datos} />;
  default: {
    const _comprobacion: never = peticion;
    return _comprobacion;
  }
}
```

Reducción de tipos

En cada case, TypeScript conoce las propiedades que existen.

Exhaustividad

Si añades un estado y no lo tratas, esa línea no compila.

TÉCNICA 6

Elegir la etiqueta: botón o enlace

```tsx
type PropsAccion =
  | { href: string; texto: string }
  | { alPulsar: () => void; texto: string };

function Accion(props: PropsAccion) {
  if ("href" in props) {
    return <a href={props.href}>{props.texto}</a>;
  }
  return (
    <button onClick={props.alPulsar}>
      {props.texto}
    </button>
  );
}
```

Dos formas, un tipo

Con href es un enlace; con alPulsar, un botón.

Operador in

Reduce el tipo: dentro del if, props ya tiene href.

Variante avanzada

Componente polimórfico con ElementType.

NOVEDAD REACT 19.2

Activity: ocultar sin perder el estado

Renderizado condicional

```
{visible && <Formulario />}
```

Al ocultarlo, el componente se desmonta: se pierde lo escrito.

Activity

```
<Activity mode={visible ? "visible" : "hidden"}>
  <Formulario />
</Activity>
```

Se oculta pero conserva el estado y el DOM; sus efectos se pausan.

> Útil en pestañas, paneles laterales y formularios por pasos.

GUÍA DE DECISIÓN

¿Qué técnica uso?

No pintar nada en todo el componente	if + return null
Mostrar u ocultar un fragmento	Operador && (con booleano)
Elegir entre dos alternativas	Operador ternario
Valor según una clave	Record<Clave, Valor>
Tres o más estados excluyentes	Unión discriminada + switch
Ocultar conservando el estado	<Activity>

BLOQUE 2

Listas

Tipar datos, recorrerlos con map y transformarlos antes de pintarlos.

PUNTO DE PARTIDA

Tipar los datos antes de pintarlos

```ts
const CATEGORIAS = ["fruta", "verdura", "lacteo"] as const;
type Categoria = (typeof CATEGORIAS)[number];

interface Producto {
  readonly id: number;
  nombre: string;
  precio: number;
  categoria: Categoria;
}

const formatearPrecio = (precio: number) =>
  new Intl.NumberFormat("es-ES", {
    style: "currency", currency: "EUR",
  }).format(precio);
```

as const

Una sola fuente de verdad: la lista y el tipo.

Intl.NumberFormat

Formato de moneda sin concatenar símbolos a mano.

IDEA CLAVE

JSX sabe pintar arrays de elementos

```
function ListaProductos() {
  const elementos: ReactElement[] = [];

  for (const producto of productos) {
    elementos.push(
      <li key={producto.id}>{producto.nombre}</li>
    );
  }

  return <ul>{elementos}</ul>;
}
```

Funciona, pero…

Es verboso: necesita array auxiliar y bucle.

Tipo ReactElement[]

Sin anotarlo, TypeScript infiere never[] y falla.

LA FORMA HABITUAL

Recorrer con map()

```
function ListaProductos() {
  return (
    <ul>
      {productos.map((producto) => (
        <li key={producto.id}>
          {producto.nombre}
        </li>
      ))}
    </ul>
  );
}
```

Inferencia

producto es Producto sin escribir el tipo.

Sin key

React avisa en consola y oxlint lo señala.

BUENA PRÁCTICA

Extraer el elemento a un componente

```tsx
interface PropsProductoItem {
  producto: Producto;
}

function ProductoItem({ producto }: PropsProductoItem) {
  return <li>{producto.nombre}</li>;
}

function ListaProductos({ productos }: Props) {
  if (productos.length === 0) return <p>Sin productos</p>;
  return (
    <ul>
      {productos.map((p) => (
        <ProductoItem key={p.id} producto={p} />
      ))}
    </ul>
  );
}
```

Props tipadas

El contrato del componente queda explícito.

Lista vacía

Combina el bloque 1 con el 2: salida temprana.

La key

Va en el elemento del map, no dentro del componente.

ESQUEMA

De los datos a la pantalla: la tubería



Estado derivado

La lista visible se calcula en cada render; no se guarda en otro estado.

Evitar

Duplicar datos en un segundo useState y sincronizarlos a mano.

ES2023

Métodos que no mutan el array

sort()	toSorted()	Ordenar
reverse()	toReversed()	Invertir
splice()	toSpliced()	Insertar o quitar
arr[i] = x	with(i, x)	Reemplazar

```
const porPrecio = productos.toSorted(
  (a, b) => a.precio - b.precio
);
```

Con sort()

Altera el array original (props o estado).

AGRUPAR Y FILTRAR BIEN

Object.groupBy y filtros con tipo

```
// agrupar por categoría
const grupos = Object.groupBy(
  productos,
  (p) => p.categoria
);
// { fruta: [...], verdura: [...] }

// quitar nulos y que TS lo entienda
const validos = datos.filter((d) => d !== null);
// validos: Producto[] (TS 5.5)
```

groupBy

Evita el reduce complicado. Las claves pueden ser parciales: usar ?.

Predicado inferido

Desde TS 5.5 ya no hace falta escribir x is Producto.

CASO PARTICULAR

Fragment con key

```
import { Fragment } from "react";

<dl>
  {productos.map((p) => (
    <Fragment key={p.id}>
      <dt>{p.nombre}</dt>
      <dd>{formatearPrecio(p.precio)}</dd>
    </Fragment>
  ))}
</dl>
```

Problema

El fragmento corto <>…</> no admite key.

Solución

Usar <Fragment key=…> importado de react.

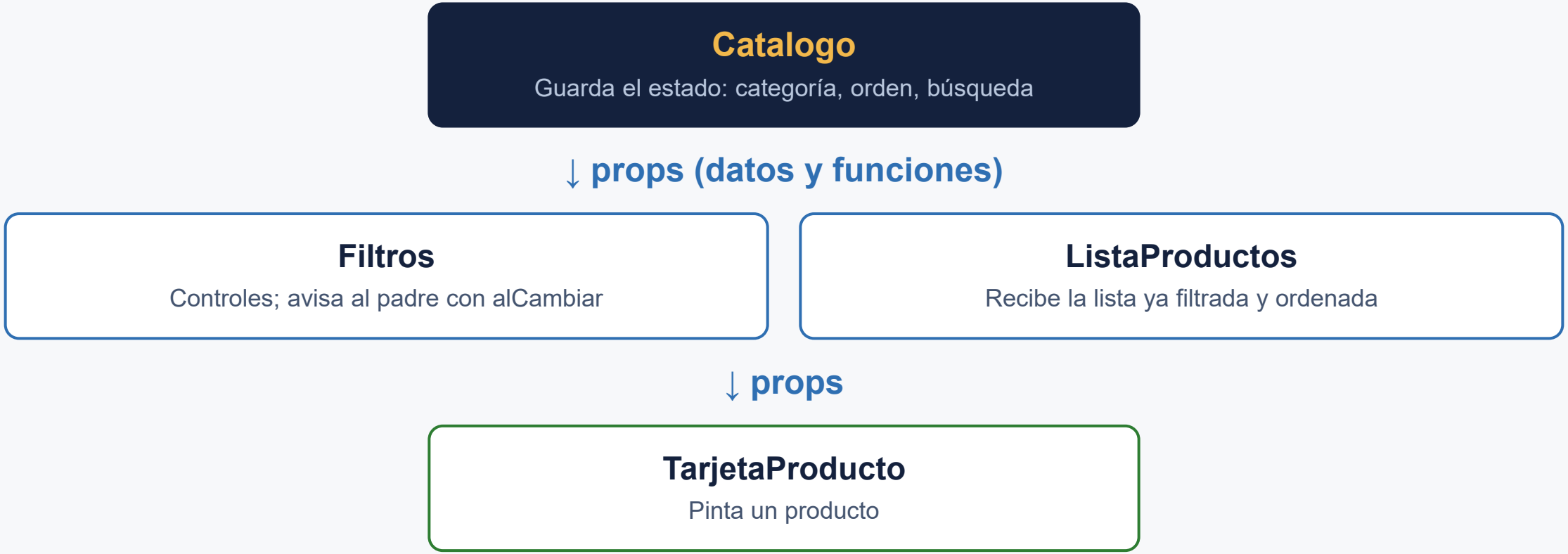
BLOQUE 3

Ejemplo integrador

Un catálogo con filtros y orden que combina todo lo visto.

ARQUITECTURA

Componentes del catálogo



CATALOGO.TSX

Estado mínimo, datos derivados

```tsx
function Catalogo() {
  const [categoria, setCategoria] =
    useState<Categoria | "todas">("todas");
  const [orden, setOrden] = useState<Orden>("nombre");

  // derivado: no es estado
  const visibles = ordenar(
    productos.filter((p) =>
      categoria === "todas" || p.categoria === categoria
    ),
    orden
  );

  return (
    <>
      <Filtros categoria={categoria} alCambiar={setCategoria} />
      <ListaProductos productos={visibles} />
    </>
  );
}
```

Solo dos estados

Categoría y orden; la lista visible se calcula.

Uniones literales

Orden y categoría no admiten valores inválidos.

ERRORES FRECUENTES

Lo que más se rompe en clase

numero && <JSX />	Aparece un 0	numero > 0 &&
map sin key	Aviso en consola	key={id}
key con el índice	Estado mezclado	Id estable
sort() sobre el estado	Orden inconsistente	toSorted()
Indexar con string	Error TS7053	Clave tipada con unión
JSX.Element global	No encuentra JSX	ReactElement / ReactNode

RESUMEN

Ideas para llevarse

Condicionales

Son JavaScript normal dentro del componente.

Usar un booleano con &&, nunca un número.

Con varios estados: unión discriminada y never.

Para conservar el estado al ocultar: Activity.

Listas

Tipar primero los datos con interface.

map() crea los elementos; cada uno con su key.

Filtrar y ordenar sin mutar: filter y toSorted.

La lista visible se deriva, no se guarda.

REPASO

Preguntas para pensar

1. ¿Qué se pinta con {0 && <p />} y por qué?

2. ¿Qué ventaja tiene Record<Pais, string> frente a un objeto sin tipar?

3. ¿Cuándo elegirías Activity en vez de &&?

4. ¿Por qué no usar sort() directamente sobre un estado?

5. ¿Qué datos del catálogo son estado y cuáles son derivados?

PRÓXIMA UNIDAD

UT 03.2 · Listas dinámicas

- La key en profundidad: índice frente a id
- Añadir, borrar y editar sin mutar el estado
- Reiniciar un componente con key
- useDeferredValue en búsquedas